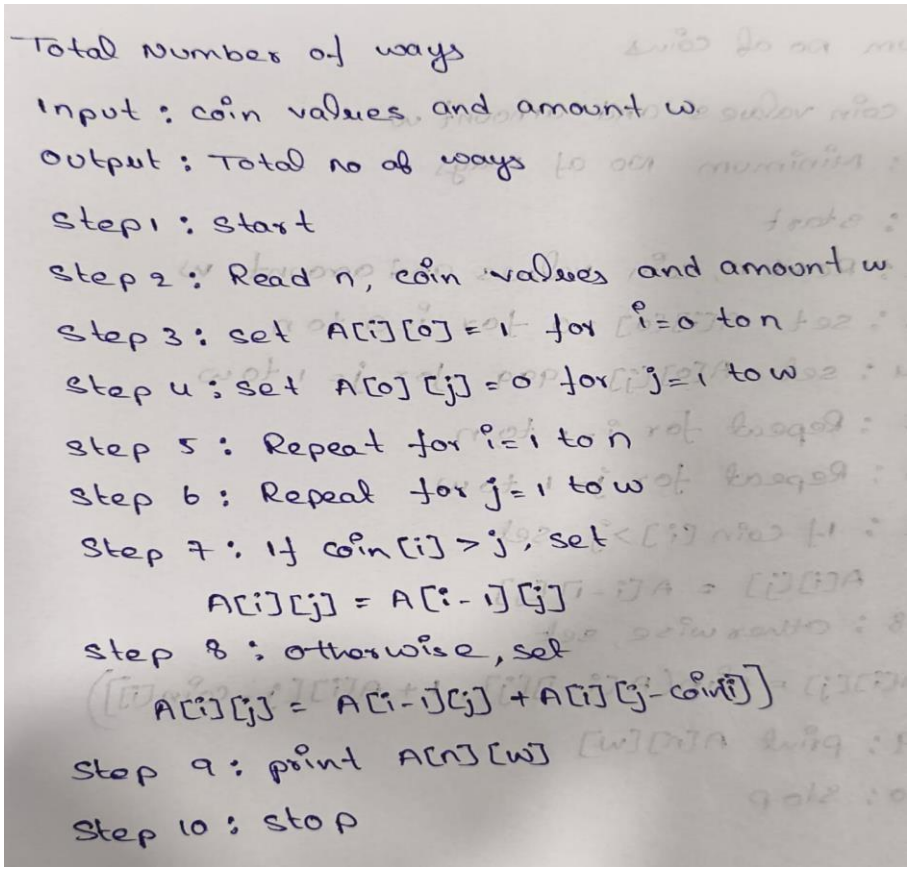
Practical- 7

Aim: To Implementation of making a change problem using dynamic programming.

Method-1: Total no of ways

Algorithm:



Program:

```
#include <iostream>
using namespace std;
class ChangeMaking {
private:
    int coins[10];
    int n;
    int w;
    int A[10][100];
public:
```

```
void getInput() {
    cout << "Enter number of coin types : ";
    cin >> n;
    cout << "Enter coin values : ";
    for (int i = 1; i <= n; i++)
        cin >> coins[i];
    cout << "Enter amount : ";
    cin >> w;
}
void solve() {
    for (int i = 0; i <= n; i++)
        A[i][0] = 1;
    for (int j = 1; j <= w; j++)
        A[0][j] = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= w; j++) {
            if (coins[i] > j)
                A[i][j] = A[i - 1][j];
            else
                A[i][j] = A[i - 1][j]
                        + A[i][j - coins[i]];
        }
    }
}
void printTable() {
    cout << "\nDP Table:\n";
    cout << "Coin ";
    for (int j = 0; j <= w; j++)
        cout << j << "   ";
    cout << endl;
    for (int i = 0; i <= n; i++) {
        if (i == 0)
            cout << "0    ";
```

```
            else
                cout << coins[i] << "    ";
            for (int j = 0; j <= w; j++)
                cout << A[i][j] << "   ";
            cout << endl;
        }
    }
    void showResult() {
        cout << "\nTotal Number of Ways = "
             << A[n][w] << endl;
    }
};
int main() {
    ChangeMaking c;
    c.getInput();
    c.solve();
    c.printTable();
    c.showResult();
    return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\practical-7\method-1.exe"
Enter number of coin types : 4
Enter coin values : 2
3
5
10
Enter amount : 15

DP Table:
Coin 0   1   2   3   4   5   6   7   8   9   10   11   12   13   14   15
0    1   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
2    1   0   1   0   1   0   1   0   1   0   1   0   1   0   1   0
3    1   0   1   1   1   1   2   1   2   2   2   2   3   2   3   3
5    1   0   1   1   1   2   2   2   3   3   4   4   5   5   6   7
10    1   0   1   1   1   2   2   2   3   3   5   4   6   6   7   9

Total Number of Ways = 9
```

Time complexity:

```
for (int i=1; i<=n; i++) {                    → 1, n+1, n
    for (int j=1; j<=w; j++) {                → n, n(w+1)
        if (coins[i] > j) {                   → nw
            A[i][j] = A[i-1][j];              → nw
        }
        else {
            A[i][j] = A[i-1][j] + A[i][j - coins[i]];
        }
    }
}
```

n = no of coin type, w = target amount

$T(n,w) = 1 + (n+1) + n + n + n(w+1) + nw + nw + nw$

$= 4nw + 4n + 2$

$= O(nw)$ → in worst case

Average case

The loops still run for every coin and every amount

$T(n,w) = O(nw)$

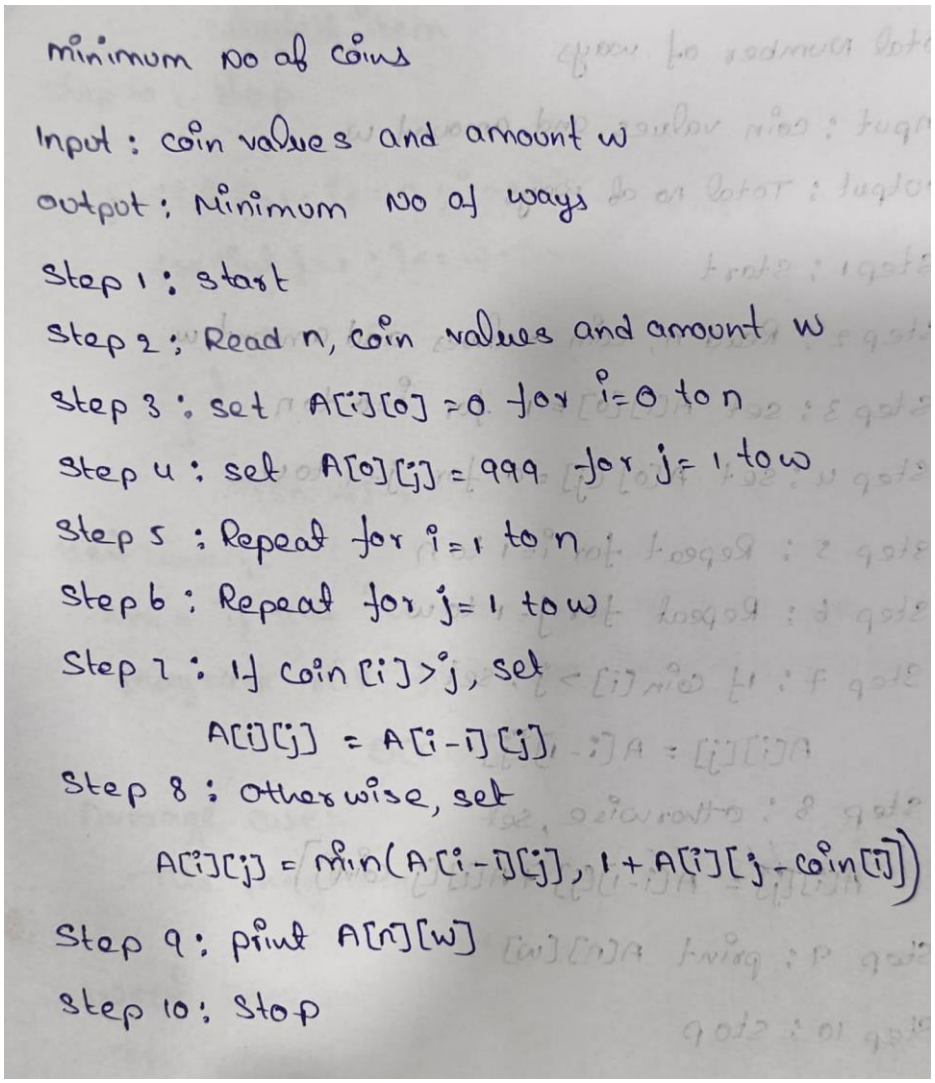
Best case :-

the complete DP table is calculated. The algorithm does not stop easly

$T(n,w) = O(nw)$

Method-2: Minimum no of ways

Algorithm:



Program:

```
#include <iostream>
#include <algorithm>
using namespace std;
class ChangeMaking {
private:
    int coins[10];
    int n, w;
    int A[100][100];
public:
    void getInput() {
```

```
    cout << "Enter number of coin types: ";
    cin >> n;
    cout << "Enter coin values: ";
    for (int i = 1; i <= n; i++){
        cin >> coins[i];
    cout << "Enter amount: ";
    cin >> w;
}
void solve() {
    for (int i = 0; i <= n; i++)
        A[i][0] = 0;
    for (int j = 1; j <= w; j++)
        A[0][j] = 999;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= w; j++) {
            if (coins[i] > j)
                A[i][j] = A[i - 1][j];
            else
                A[i][j] = min(A[i - 1][j],
                              1 + A[i][j - coins[i]]);
        }
    }
}
void printTable() {
    cout << "\nDP Table:\n";
    cout << "      ";
    for (int j = 0; j <= w; j++)
        cout << j << "   ";
    cout << endl;
    for (int i = 0; i <= n; i++) {
        cout << "i=" << i << "  ";
        for (int j = 0; j <= w; j++) {
```

```
            if (A[i][j] == 999)
                cout << "0   ";
            else
                cout << A[i][j] << "   "; }
        cout << endl;  } }
    void showResult() {
        if (A[n][w] == 999)
            cout << "\nAmount cannot be formed.";
        else
            cout << "\nMinimum number of coins = "<< A[n][w];
    }
};
int main() {
    ChangeMaking c;
    c.getInput();
    c.solve();
    c.printTable();
    c.showResult();
    return 0;
}
```

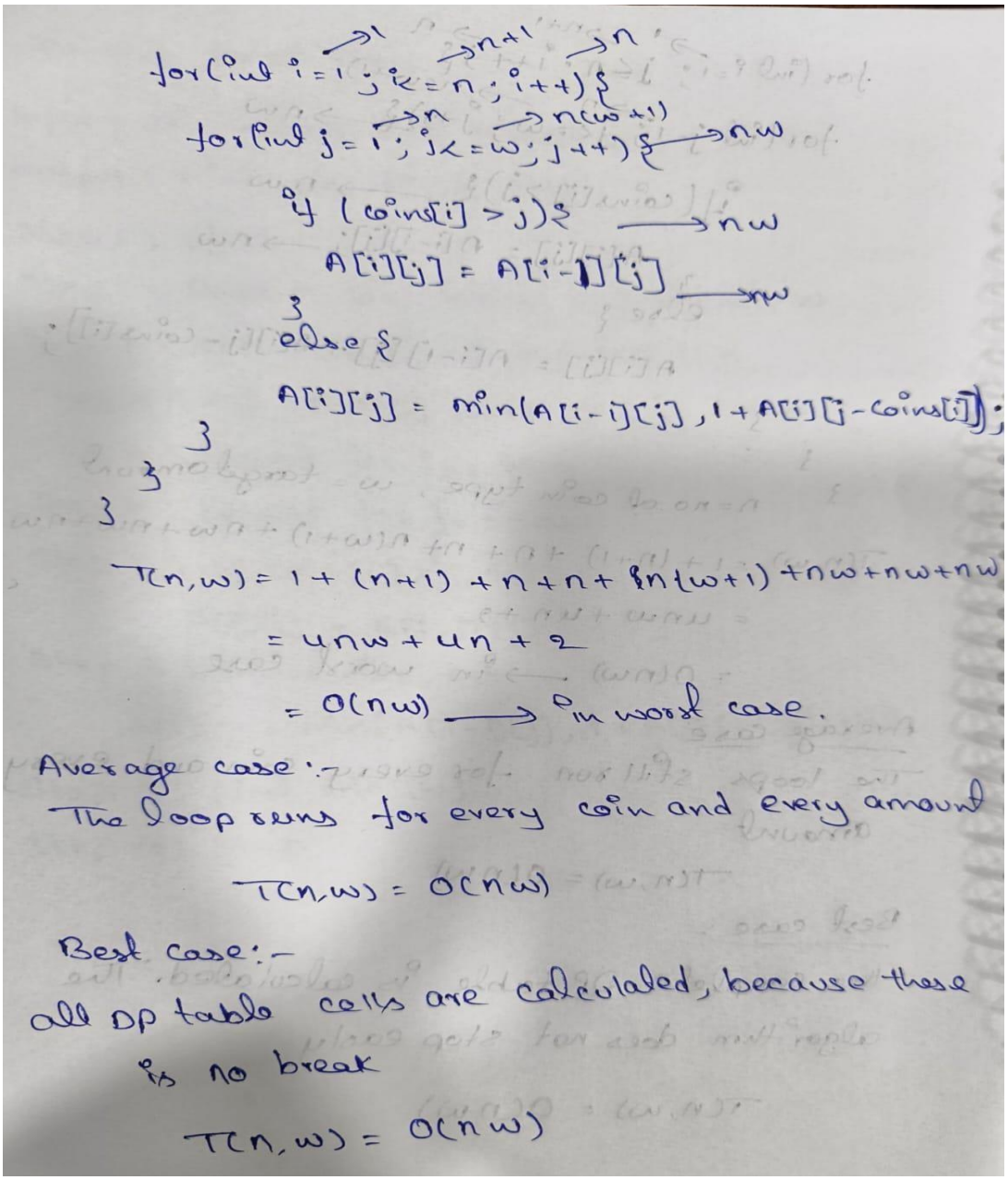
Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\practical-7\method-2.exe"
Enter number of coin types: 4
Enter coin values: 1
5
6
9
Enter amount: 10

DP Table:
      0   1   2   3   4   5   6   7   8   9   10
i=0   0   0   0   0   0   0   0   0   0   0   0
i=1   0   1   2   3   4   5   6   7   8   9   10
i=2   0   1   2   3   4   1   2   3   4   5   2
i=3   0   1   2   3   4   1   1   2   3   4   2
i=4   0   1   2   3   4   1   1   2   3   1   2

Minimum number of coins = 2
```

Time complexity:



Conclusion: The Making Change problem was successfully implemented using Dynamic Programming. The algorithm stores the solutions of smaller subproblems in a DP table and uses them to find the required solution efficiently. It reduces repeated calculations and provides an efficient approach for finding the minimum number of coins or total number of ways to make a given amount.

Name: S.jaswanth siva kumar
Enrollment Number:92460118154